TECHNICAL ASSESSMENT

AgentSight + eBPF Implementation Guide for Copilot

Repository baseline: xsaidi1992/preemptics-test

Goal: turn the current prototype into a defensible, reproducible submission for the Backend / Systems Engineering assessment, while preserving useful existing code and explicitly closing the gaps against the assignment.

COPILOT OPERATING RULE

Inspect the repository before editing. Do not invent files, APIs, AgentSight internals, performance numbers, or successful eBPF behavior. Reuse existing components when they are correct; replace simulations only where they prevent an end-to-end proof. Every claim in README must be backed by code and a reproducible command.

Prepared as an execution brief, not as a final submission. The resulting repository must contain the actual source code, tests, README, architecture explanation, and live demonstration evidence required by the assessment.

Priority order

1. Live kernel-to-user-space capture -> 2. Process/session correlation -> 3. Sensitive-action detection -> 4. API -> 5. LLM/OS correlation -> 6. Persistence, tests and documentation

1. What the assessment actually requires

The supplied assessment asks for a minimal OS-level security sensor for an AI agent, using AgentSight and eBPF, without modifying the monitored agent application. The submission must show the path from kernel observation to an inspectable security event, not merely simulate that path.

Part	Requirement	Definition of done
A	Architecture	Explain Linux kernel -> eBPF probe -> ring/perf buffer -> user-space collector -> event model -> correlation -> storage/export -> AgentSight/report. Identify the relevant AgentSight components.
B	eBPF probe	Capture process execution and emit at least timestamp, pid, ppid, uid, comm, executable and command. Demonstrate child-process association.
C	Agent session	Represent an agent session with root PID/start/command/parent plus processes/files/network events; explain lineage attribution.
D	Sensitive action	Generate a security event for at least one sensitive command or path. Detection quality and explainability matter more than blocking.
E	LLM <-> OS	Show how LLM activity is correlated with subsequent process/file/network activity, and state limitations.
F	Backend API	Expose agent list/details/timeline/processes/security-events plus filtered event search.
Perf	Load behavior	Discuss filtering, ring/perf buffer, batching, backpressure, sampling/aggregation, memory/CPU and event-loss observability.
Deliver	Evidence	Source, README, architecture diagram, design rationale, reproducible demo, automated tests, limitations and production improvements.

NON-NEGOTIABLE

A Python test that manually constructs ProcessExecutionEvent/FileAccessEvent objects is useful as

a unit test, but it does not satisfy the live eBPF demonstration. At least one automated or scripted demo must originate from a real Linux process event observed by the kernel probe.

2. Existing project baseline: preserve what works, close what is missing

The current repository already contains useful building blocks: Python models, collector/session logic, a FastAPI layer, security-rule logic, an eBPF C probe prototype, and demonstration/tests. The key review issue is that the current evidence is stronger for simulated event flow than for a complete live eBPF path. The implementation should therefore evolve the current design rather than start over.

Area	Decision	Required action
Event models	KEEP / tighten	Retain Pydantic/domain models if they already carry the required fields. Add explicit source, monotonic/kernel timestamp, event sequence and optional exit/result fields.
SessionManager / correlation	KEEP / harden	Preserve current PID/PPID correlation; make it deterministic for descendants, late events and process exit. Add root_pid and lineage map.
SecurityEngine	KEEP / make declarative	Retain existing sensitive-command/path logic; ensure each finding includes rule_id, reason/evidence, severity, session_id and event reference.
FastAPI server	KEEP / align routes	Keep the API but add the exact per-agent routes from the assessment and consistent filtering/pagination.
src/ebpf/probe.c	MODIFY	Treat it as the starting kernel probe. Ensure it builds and is actually loaded. Capture argv sufficiently for command reconstruction and emit via ring buffer.
Main/demo simulation	KEEP only as fallback/unit demo	Do not present simulated events as the primary proof. Add a live mode that consumes real probe events.
Persistence	IMPLEMENT or verify	Use SQLite or JSONL. If README claims persistence already exists, make code and paths match the claim.

Area	Decision	Required action
AgentSight linkage	IMPLEMENT/document	Explicitly map reused AgentSight concepts/components and, where practical, consume AgentSight output or adapt its process capture design. Do not claim code reuse unless actual code is reused.

3. Target architecture

```
Monitored AI agent process
        |
        | execve / fork / file / network activity
        v
+---------------------- Linux kernel ----------------------+
| eBPF process probe (minimum: exec)                        |
|   - PID / PPID / UID / comm / executable / argv           |
|   - timestamp + sequence                                  |
|   - optional kernel-side PID filter                       |
+---------------------------+-------------------------------+
                            | BPF ring buffer
                            v
                 +------------------------+
                 | user-space probe runner|
                 | validate/decode events |
                 +-----------+------------+
                             |
                             v
+------------------------ Python runtime --------------------+
| EventNormalizer -> SessionManager -> SecurityEngine         |
|         |                 |                 |               |
|         +----------> SQLite / JSONL <------+               |
|                           |                               |
|                       FastAPI API                          |
+-----------------------------+-------------------------------+
                              |
                              v
                    timeline / report / demo

AgentSight integration/reference path:
AgentSight process/LLM capture or exported session data
        -> AgentSightAdapter -> normalized timeline -> same correlation/reporting layer
```

Design principle: keep a single canonical event model after ingestion. eBPF live events, AgentSight-derived events and test fixtures should all be normalized before correlation; downstream security and API code should not care which transport produced them.

4. Implementation plan by repository area

4.1 First task for Copilot: repository inventory and gap report

Before changing code, Copilot should produce a short inventory in the working tree or terminal output. This prevents the README/code drift already observed during review.

```
Inspect and report:
- actual tree under src/, tests/, scripts/, docs/
- all models and their fields
- current SessionManager lineage algorithm
- current SecurityEngine rules/output
- current FastAPI routes
```

```
- current eBPF program sections/maps/event struct
- whether any loader actually compiles/loads/attaches the BPF object
- whether persistence exists in code (not only README)
- exact test commands and which tests are simulation-only vs live
- all README claims that are not backed by current code
```

4.2 Kernel probe: make the eBPF path real

Preferred approach: preserve the existing libbpf-style probe if viable and add the missing build/load/consume path. Avoid rewriting the project around BCC unless the existing probe cannot be made operational in the target environment.

- Use a process-execution hook that is explainable and stable. If keeping sched_process_exec, document that it represents successful exec and how argv is obtained. If switching to sys_enter_execve, document that it observes attempts and optionally pair with sys_exit_execve to record success/failure.
- Event structure must include: monotonic timestamp (ns), pid/tgid as appropriate, ppid, uid, comm, executable, command/argv, sequence number, and event type/version.
- Use BPF ring buffer (preferred) for kernel-to-user communication. Reserve -> populate -> submit; increment a sequence counter so userspace can detect gaps.
- Read PPID from the current task/real_parent using CO-RE when possible. Verify PID namespace assumptions and document them.
- Bound all strings and argv capture. Truncate safely and set a TRUNCATED flag; never assume arbitrary argv length can fit in one event.
- Do not perform expensive security policy string matching in kernel for the first version. Optional root-PID/UID filtering is acceptable for load reduction.
- On userspace decode failure, count and log malformed events; do not crash the entire collector.

4.3 Loader / user-space collector

The current Python architecture needs a concrete bridge from the eBPF object to Python. Choose one path and implement it completely; do not leave a placeholder loader.

Path	Mechanism	Assessment value
Recommended	Small libbpf C runner + JSONL/stdout or Unix socket	Best fit if existing probe is libbpf CO-RE. Build BPF object and a tiny runner, poll ring buffer, serialize normalized JSON; Python launches/reads it.
Alternative	Native Rust helper / AgentSight capture adapter	Good if directly reusing AgentSight collector code is practical; expose a stable JSON or SQLite boundary to Python.
Avoid	A fake Python generator called “eBPF loader”	Does not prove kernel capture and will be obvious in review.

Suggested repository additions (adapt names to the actual tree after inspection):

```
src/ebpf/
  process_exec.bpf.c          # kernel-side program (or evolve existing probe.c)
  process_exec.h              # shared event ABI if using C runner
```

```
  process_exec_user.c          # libbpf loader + ring-buffer poller
  Makefile                     # clang/bpftool/libbpf build
src/collector/
  ebpf_collector.py            # process lifecycle, JSONL decode, counters
  event_normalizer.py          # ABI -> domain model
src/runtime/
  orchestrator.py              # optional: wire collector/session/security/storage
scripts/
  demo_live.sh
  check_ebpf_env.sh
```

4.4 Process-tree and session attribution

This is central to the exercise. Correlation must not be “same PID only”. Track process lineage so that bash, curl, git, python, etc. spawned below the agent root remain in the same session.

```
Recommended state:
process_to_session: dict[int, session_id]
parent_by_pid:      dict[int, int]
known_processes:    dict[int, ProcessInfo]

on_exec(event):
    if event.pid == configured_agent_root_pid:
        bind(event.pid -> session_id)
    elif event.ppid in process_to_session:
        bind(event.pid -> process_to_session[event.ppid])
    else:
        walk bounded parent chain / consult known lineage

on_fork(optional):
    inherit parent session immediately, before child exec

on_exit(optional):
    mark process ended; retain lineage metadata for historical queries
```

If only exec events are captured, child attribution works when PPID is still a known tracked process. Capturing fork/clone and exit events is a strong improvement because it closes timing gaps and handles children before exec.

4.5 Security rule

Implement at least one deterministic rule and make the evidence explicit. A sensitive-command rule is the easiest end-to-end proof because it can be triggered from the process probe alone.

```
Rule SENSITIVE_COMMAND_EXEC
match executable basename or argv[0] in:
  curl, wget, ssh, sudo, chmod, rm

SecurityEvent:
{
  "type": "AI_AGENT_SECURITY_EVENT",
  "rule_id": "SENSITIVE_COMMAND_EXEC",
  "severity": "HIGH",
  "session_id": "...",
  "pid": 4312,
  "action": "PROCESS_EXECUTION",
  "target": "/usr/bin/curl",
  "reason": "Agent session executed command 'curl'",
  "source_event_id": "...",
  "timestamp": "..."
}
```

Optional second rule: sensitive path access (`/etc/passwd`, `/etc/shadow`, `~/.ssh/`, `.env`). Only include it as a claimed feature if a real file-access probe and live demonstration exist.

4.6 Persistence and event-loss telemetry

- Persist normalized events and security events to SQLite using WAL mode, or append JSONL if time is extremely constrained. SQLite is preferable because the assessment API needs filtered queries.
- Persist sessions separately from events. Minimum tables: sessions, events, security_events; optional processes table for efficient lineage queries.
- Track collector counters: received, decoded, dropped_by_sequence_gap, malformed, storage_failures, queue_depth/high_watermark.
- Write asynchronously/batched (for example batches of 100 or 100 ms) so DB latency does not block ring-buffer polling.
- If the userspace queue is full, use an explicit policy (drop newest or oldest), increment a counter, and surface it in `/health` or `/statistics`.

4.7 API: match the assignment exactly

Keep existing endpoints for compatibility, but add or align the following routes so the evaluator can map the implementation directly to Part F:

```
GET /agents
GET /agents/{id}
GET /agents/{id}/timeline
GET /agents/{id}/processes
GET /agents/{id}/security-events
GET /events?pid=4312
GET /events?severity=HIGH

Recommended extras:
GET /health            # probe attached? collector alive? drops? storage?
GET /statistics        # event totals, security counts, drop counters
```

Return stable JSON schemas. Add `limit`, `offset` (or cursor) to high-volume event routes. Invalid session IDs should be 404; invalid filters 422/400, not empty 200 responses that hide errors.

4.8 LLM-to-OS correlation using AgentSight

The assessment explicitly asks to use AgentSight capabilities as the basis of LLM correlation. The safest implementation is an adapter rather than inventing semantic prompt inference.

- Document which AgentSight component/source provides LLM request/model/tool activity and which provides process/system events. AgentSight currently exposes live/record workflows and saved SQLite sessions; use its real output format after inspecting the version used in the environment.
- Implement `AgentSightAdapter` that converts AgentSight LLM/process records into the project canonical event model. Keep the adapter tolerant to missing optional fields.
- Correlate primarily by process/session identity and timestamps, not by “understanding” prompt text. Record correlation confidence/method (`same_process_family`, `within_time_window`, `explicit_agent_session`).
- For the demonstration, prefer `agentsight record -- <agent command>` when the environment supports it. If a real provider API key is unavailable, keep the OS demo fully offline and include a sanitized, clearly identified AgentSight capture fixture for the LLM-correlation test; do not pretend a synthetic fixture was captured live.

IMPORTANT FOR REVIEW

AgentSight is not just a name to mention in README. The submission should contain a concrete mapping of AgentSight source/components to this implementation and a transparent statement of what is reused, adapted, or independently implemented.

5. Reproducible demonstration: the evidence the reviewer should see

Create one command that proves the critical path on a Linux eBPF-capable host. It should fail clearly if kernel privileges or build prerequisites are missing, rather than silently falling back to simulation.

```
# Example target UX (adapt to actual CLI)
make ebpf
sudo ./scripts/demo_live.sh

# demo_live.sh should:
# 1. start API/collector with a new session root
# 2. run a tiny demo agent process
# 3. demo agent spawns: /bin/sh -> /usr/bin/curl --version
# 4. wait for collector flush
# 5. query API and print:
#    - process tree proving child attribution
#    - timeline with kernel-originated exec events
#    - HIGH AI_AGENT_SECURITY_EVENT for curl
#    - collector received/drop counters
# 6. exit non-zero if any assertion fails
```

Use `curl --version` (or another harmless sensitive command) so the security rule is triggered without external network access. The important proof is that `/usr/bin/curl` was actually executed as a child and observed through the kernel probe.

6. Test strategy

Level	Test	Acceptance criterion
Unit	Event normalization	Decode known raw event -> canonical model; truncation and malformed input cases.
Unit	Session lineage	root -> shell -> curl inherits one session; unrelated PID does not.
Unit	Security engine	curl triggers HIGH; harmless command does not; rule output has evidence.
Unit	Persistence	write/read session/events; filters; ordering.
API	FastAPI	required routes, 404s, pid/severity filters, timeline ordering.
Integration	Live eBPF	On Linux with privileges: launch child process and assert the event

Level	Test	Acceptance criterion
		came from the real collector. Mark/skip with an explicit reason when unavailable.
Integration	End-to-end security	Live demo process tree produces security event and API exposes it.
Fixture	AgentSight correlation	Parse a real sanitized AgentSight export/DB fixture and correlate an LLM event to subsequent OS activity.

Do not label a test “eBPF integration” if it bypasses the ring buffer or constructs domain events directly. Keep simulation tests, but name them accurately (`unit`, `synthetic`, or `fixture`).

7. Performance and failure-mode section to implement and document

Concern	Required design/documentation
Kernel filtering	Optional filter by root PID/UID/cgroup after correctness is proven. Avoid expensive path policy in kernel for v1.
Ring buffer	Non-blocking kernel emission; reservation failure/drop counters where possible; sequence gaps observable in userspace.
Batching	Decouple ring polling from SQLite writes with bounded queue and periodic batches.
Backpressure	Never let DB/API work block the ring-buffer consumer. Define bounded queue and explicit drop behavior.
Sampling	Do not sample security-sensitive exec events in the assessment demo. Sampling may be a production option for noisy low-value event types.
Memory	Bound argv, event size, queues, retained in-memory sessions and API page size.
CPU	Measure rather than claim. Provide a simple benchmark command and report the machine/kernel used.
Loss visibility	Expose sequence gaps, ring reserve failures if available, userspace queue drops, malformed count and storage failures.

8. README structure the final repository should contain

1. 1. Problem statement and scope (observation/detection, not blocking).
2. 2. Architecture diagram and end-to-end data path.

3. 3. AgentSight mapping: components studied/reused/modified/implemented independently.
4. 4. eBPF design: hook, event ABI, ring buffer, argv limits, PPID/process-tree tracking, loss/error handling.
5. 5. Prerequisites: Linux kernel, clang/LLVM, libbpf/bpftool, Python, sudo/capabilities; exact supported environment.
6. 6. Build and run commands from a clean checkout.
7. 7. Reproducible live demo with expected JSON excerpts.
8. 8. API reference with the exact required endpoints.
9. 9. Tests: unit vs live integration clearly separated.
10. 10. Performance/load discussion and measurable counters.
11. 11. Limitations and production roadmap (namespaces, containers, PID reuse, argv truncation, multi-worker, retention, authentication).

9. Definition of done / reviewer checklist

☐ Fresh clone builds using documented commands on the stated Linux environment.

☐ BPF program compiles and attaches; startup logs identify the actual hook(s).

☐ Executing a real child command generates a kernel-originated event visible in userspace.

☐ Event includes timestamp, pid, ppid, uid, comm, executable and command/argv equivalent.

☐ Root agent + descendants are shown in one session/process tree.

☐ A harmless `curl --version` child execution produces a HIGH security event.

☐ API exposes `/agents/{id}/timeline`, `/processes`, `/security-events` plus filtered `/events`.

☐ At least one live integration test/demo is distinct from synthetic unit tests.

☐ Event-loss and collector health counters are visible.

☐ AgentSight reuse/reference is concrete and version-aware; no invented claims.

☐ README claims match the checked-in tree and runnable behavior.

☐ Limitations are explicit; no unsupported “production-ready” language.

10. Suggested Copilot execution prompt

Give Copilot the repository plus this PDF and use the following as the top-level instruction. It is intentionally strict to reduce broad rewrites and fabricated completion claims.

```
You are implementing the attached Preemptics AgentSight/eBPF technical assessment in THIS
existing repository.

Rules:
1. Inspect the repository before editing and write a short gap inventory against the
assessment.
2. Preserve working models, session correlation, security rules and FastAPI code where
possible.
3. The primary requirement is a REAL Linux eBPF -> ring buffer -> userspace -> session ->
security event -> API path. Do not satisfy this with synthetic Python events.
4. Treat the existing eBPF probe as a baseline. Make it build, attach and emit the required
process execution fields. Add a real loader/consumer.
5. Implement deterministic child-process attribution to the root agent session.
6. Implement at least SENSITIVE_COMMAND_EXEC and demonstrate it harmlessly with `curl --
version` as a child process.
```

```
7. Align the API with the exact endpoints in the assessment.
8. Add SQLite/JSON persistence and bounded, observable event ingestion. Surface event-loss
counters.
9. Study the current upstream AgentSight version. In README identify exact components used as
references and what code is reused, adapted or independent. Implement a concrete
adapter/correlation path for AgentSight LLM/session data if practical.
10. Add unit tests and a Linux-only live integration test. Never call a synthetic test “eBPF
integration”.
11. Create `scripts/demo_live.sh` that proves the live end-to-end path and exits non-zero if
assertions fail.
12. Update README only after implementation. Remove or correct any claim that is not backed by
source code and a reproducible command.
13. Do not claim performance percentages without a benchmark performed in this repository.
14. Keep the scope minimal and assessment-focused; no UI rewrite and no premature policy
blocking.

Work in small commits/steps. After each major step run tests/builds and report exact command
output. If the environment lacks Linux/eBPF privileges, implement the code but explicitly mark
the live verification as NOT RUN and provide the exact command to run on a suitable host.
```

11. Evidence package to include before submission

- `README.md` with architecture and precise build/run/test/demo steps.
- One architecture diagram committed as Mermaid or ASCII (and rendered image optional).
- `scripts/demo_live.sh` plus a captured sample output (`docs/demo-output.txt` or sanitized JSON).
- Test report showing unit tests and the separate live eBPF integration result.
- A short `docs/agentsight-mapping.md` that points to actual upstream AgentSight directories/components used in the chosen version.
- A short `docs/limitations.md` or README section describing PID namespaces/reuse, argv truncation, privileges, container behavior, event loss, storage retention and LLM-correlation ambiguity.

12. Sources used for this implementation brief

Assessment source: “Technical Assessment — AI Agent OS-Level Monitoring with eBPF & AgentSight” supplied with this task (5 pages). Core requirements include live process execution capture, agent-session/process-tree correlation, sensitive-action detection, LLM/OS correlation, backend endpoints, load/loss discussion, and reproducible evidence.

AgentSight upstream (reviewed 15 Aug 2026): GitHub repository `eunomia-bpf/agentsight` and official documentation at `eunomia.dev/agentsight/`. Current public documentation describes a Rust collector, eBPF capture, process/file/network activity, saved SQLite sessions, `record`, `report`, `top`, and direct process tracing. Copilot must inspect the exact checked-out upstream version before relying on internal filenames or schemas.

Existing repository baseline: `https://github.com/xsaidi1992/preemptics-test.git`, as reviewed in this conversation. Because repository contents can change, Copilot must treat its local checkout as authoritative and reconcile this guide with the actual tree before edits.